

Relações entre leitura, traçado e escrita

Habilidades em Programação Introdutória

Mike Lopez
Instituto de Tecnologia de Manukau
Caixa Postal 94006
Centro de Correios de South Auckland
Manukau 2240
Nova Zelândia
+64 9 968 8000
mike.lopez@manukau.ac.nz

Jacqueline Whalley Phil
Robbins Auckland
University of Technology Private Bag 92006
Auckland 1020 Nova
Zelândia +64 9 921
9999
jacqueline.whalley@aut.ac.nz
phil.robbins@aut.ac.nz

Raymond Lister
Universidade de Tecnologia de Sydney,
Postal 123,
Broadway, NSW 2007,
Austrália,
+61 2 9514 2000
raymond@it.uts.edu.au

RESUMO1

Este estudo analisou as respostas dos alunos a um exame, após a conclusão de um semestre de instrução em programação. O desempenho dos alunos em tarefas de rastreamento de código correlacionou-se com seu desempenho em tarefas de escrita de código. Também foi encontrada uma correlação entre o desempenho em tarefas de "explicação em linguagem simples" e a escrita de código. Uma regressão stepwise, com o desempenho na escrita de código como variável dependente, foi utilizada para construir um diagrama de caminhos. O diagrama sugere a possibilidade de uma hierarquia de tarefas relacionadas à programação. O conhecimento de construções de programação forma a base da hierarquia, com "explicação em linguagem simples", quebra-cabeças de Parsons e o rastreamento de código iterativo formando um ou mais níveis intermediários na hierarquia.

Categorias e Descritores de Assunto

K.3 [Computadores e Educação]: Educação em Ciência da Computação e Informação - *Educação em Ciência da Computação*.

Termos Gerais

Medição, Experimentação, Fatores Humanos.

Palavras-chave

Programadores iniciantes, CS1, compreensão, taxonomia SOLO.

1. INTRODUÇÃO A década de

1980 foi um período de extenso estudo sobre programadores iniciantes. Para citar apenas dois dos muitos estudos daquela época, Soloway et al. (1983) descobriram que apenas 38% dos estudantes iniciantes em programação de computadores conseguiam escrever um programa para calcular a média de um conjunto de números, enquanto Perkins e Martin (1989) relataram que os alunos tinham um conhecimento frágil de

conceitos básicos de programação e uma "deficiência em estratégias elementares de resolução de problemas". No final dessa década, um volume inteiro de artigos, *"Estudando o Programador Iniciante"*, documentou muitas das dificuldades de aprender a programar (Soloway e Spohrer, 1989).

Na virada do milênio, um grupo de trabalho do ITiCSE 2001, o "Grupo McCracken", avaliou a capacidade de programação de um grande grupo de estudantes de quatro universidades em dois países (McCracken et al., 2001). Cada estudante deveria escrever um programa a partir de um conjunto de tarefas relacionadas à programação. Quase todos os estudantes tiveram um desempenho ruim na tarefa e muitos sequer chegaram perto de concluí-la. A natureza do experimento não permitiu que o Grupo McCracken chegasse a conclusões definitivas sobre os motivos das dificuldades dos estudantes, mas eles especularam que isso se devia à fraca capacidade de resolução de problemas. Ou seja, conforme a definição de resolução de problemas adotada pelo Grupo McCracken, os estudantes apresentavam dificuldades em um processo iterativo de cinco etapas: (1) Abstrair o problema a partir de sua descrição, (2) Gerar subproblemas, (3) Transformar subproblemas em subsoluções, (4) Recompilar e (5) Avaliar e iterar.

Na conferência ITiCSE 2004, realizada em Leeds, o único grupo de trabalho daquele ano (daí o nome "Grupo de Leeds") conduziu um experimento com o objetivo de contestar a hipótese de que os resultados do Grupo McCracken se deviam à dificuldade dos alunos em resolver problemas (Lister et al., 2004). O Grupo de Leeds estudou o desempenho dos alunos em tarefas de programação que não exigiam resolução de problemas. Os alunos deveriam responder a várias questões de múltipla escolha. As questões eram de dois tipos: questões de "código fixo" e questões de "código esqueleto". Nas questões de código fixo, os alunos recebiam um trecho de código e precisavam identificar o valor de uma variável após a execução completa do código. Responder a esse tipo de questão exige que o aluno compreenda todas as estruturas do código e também que seja capaz de executar o código manualmente ("rastrear") de forma sistemática. Nas questões de código esqueleto, os alunos recebiam um trecho de código com uma ou duas linhas faltando, também era informado o que o código deveria fazer e, em seguida, eles precisavam identificar as linhas de código faltantes (como a questão era de múltipla escolha, os alunos não precisavam escrever as linhas, mas apenas identificar o código correto entre quatro opções). Embora muitos alunos tenham se saído bem nessas questões, aproximadamente 25%

É concedida permissão para fazer cópias digitais ou impressas, totais ou parciais, desta obra para uso pessoal ou em sala de aula, sem custo, desde que as cópias não sejam feitas ou distribuídas para fins lucrativos ou comerciais e que as cópias contenham este aviso e a citação completa na primeira página. Para copiar de outra forma, republicar, postar em servidores ou redistribuir para listas, é necessária permissão específica prévia e/ou o pagamento de uma taxa.
ICER'08, 6 a 7 de setembro de 2008, Sydney, Austrália.
Direitos autorais 2008 ACM 978-1-60558-216-0/08/09...\$5,00.

dos alunos apresentaram um desempenho compatível com o palpite. Nas considerações finais do relatório do grupo de trabalho (Lister et al., 2004), o Grupo de Leeds escreveu: *"Reconhecemos que um aluno que obtém*

uma boa pontuação nos tipos de testes utilizados neste estudo, mas que não consegue escrever código inédito de complexidade semelhante, provavelmente apresenta uma dificuldade na resolução de problemas. Este grupo de trabalho apenas observa que qualquer projeto de pesquisa que vise estudar as habilidades de resolução de problemas em programadores iniciantes deve incluir um mecanismo para identificar os participantes com dificuldades em habilidades precursoras relacionadas à leitura."

Nesta observação final, o grupo de Leeds posiciona a capacidade de ler código (de uma determinada complexidade) como uma habilidade precursora da capacidade de escrever código (de complexidade semelhante). Ao postular a existência de tal habilidade precursora, o grupo de Leeds abriu a possibilidade de haver uma hierarquia multinível de habilidades de programação. Os dados do grupo de Leeds já indicam dois níveis possíveis abaixo da escrita de código, visto que os alunos tiveram menos sucesso nas questões de código-esqueleto naquele estudo do que nas questões de código fixo.

O projeto BRACElet, desde então, baseou-se nos resultados do Grupo de Leeds, investigando níveis intermediários em uma hierarquia de habilidades relacionadas à programação (Whalley et al., 2006). O instrumento de pesquisa inicial, desenvolvido pelos membros do BRACElet, reproduziu algumas das questões de código fixo do Grupo de Leeds. O instrumento também continha outras questões elaboradas por uma abordagem mais sistemática e teoricamente fundamentada do que os métodos ad hoc utilizados pelo Grupo de Leeds. A taxonomia de Bloom revisada (Anderson et al., 2001) e a taxonomia SOLO (Biggs e Collis, 1982) foram utilizadas pelos participantes do BRACElet como uma estrutura cognitiva para atribuir níveis de dificuldade às tarefas do instrumento (Whalley, Clear e Lister, 2007; Whalley e Robbins, 2007; Thompson et al., 2008). A partir dos dados coletados neste estudo inicial do BRACElet, constatou-se que os alunos consideraram as tarefas dos níveis mais altos da estrutura mais difíceis do que as tarefas atribuídas aos níveis mais baixos (Whalley, et al., 2006).

Neste primeiro estudo com BRACElet, uma das tarefas exigia que os alunos *explicassem, em linguagem simples, o que o seguinte segmento de código Java fazia*. As respostas dos alunos foram analisadas de acordo com a taxonomia SOLO. Constatou-se que alguns alunos responderam com uma descrição correta, linha por linha, do código (o que, em termos de SOLO, caracteriza uma resposta multiestrutural), enquanto outros responderam com um resumo correto do cálculo geral realizado pelo código (o que, em termos de SOLO, caracteriza uma resposta relacional). Além disso, quanto melhor o desempenho do aluno em outras tarefas com BRACElet, maior a probabilidade de ele fornecer uma resposta relacional à pergunta "explique em linguagem simples".

Além disso, quando a mesma pergunta "explique em linguagem simples" foi feita a acadêmicos, eles quase sempre ofereceram uma resposta relacional. O grupo BRACElet (Lister et al., 2006) concluiu que a capacidade de ler um trecho de código e explicá-lo em termos relacionais — ou seja, enxergar a floresta e não apenas as árvores — é uma habilidade intermediária na hierarquia de habilidades relacionadas à programação.

A conclusão deles é consistente com a literatura anterior sobre a psicologia da programação (Adelson 1984, Corritore e Weidenbeck 1991, Wiedenbeck, Fix e Scholtz 1993).

Philpott, Robbins e Whalley (2007) apresentaram dados adicionais sugerindo que o rastreamento de código é uma habilidade precursora do pensamento relacional em perguntas do tipo "explique em linguagem simples":

"O sinal verde para o pensamento relacional parece ser o domínio completo da tarefa de rastreamento de código. Melhor do que

Um desempenho de 50% na tarefa de traçado pode ser visto como um sinal laranja. No entanto, se a habilidade de traçado for inferior a 50%, o sinal é definitivamente vermelho quando se trata de pensamento relacional.

1.1 Nosso Estudo O

objetivo final da pesquisa que investiga programadores iniciantes é aprimorar a prática do ensino de programação para esses alunos. No entanto, a relação entre pesquisa e prática não precisa ser unilateral, da pesquisa para a prática. Com o devido preparo, as atividades práticas de ensino também podem servir como atividades de coleta de dados para a pesquisa. Embora a coleta de dados do Grupo de Leeds tenha sido realizada principalmente de forma independente das atividades de ensino, as questões de múltipla escolha utilizadas naquele estudo foram todas retiradas de provas aplicadas anteriormente por um dos participantes do projeto. Além disso, a maior parte dos dados coletados nos estudos BRACElet provém de questões incorporadas às provas de final de semestre.

Neste artigo, analisamos um exame de final de primeiro semestre aplicado a alunos de uma única instituição. Algumas das questões deste exame foram elaboradas com base nos resultados dos estudos do grupo de Leeds e do BRACElet. Do ponto de vista da pesquisa, o exame teve como objetivo investigar mais a fundo a noção de uma hierarquia de habilidades relacionadas à programação. As questões de pesquisa específicas abordadas neste estudo são:

- A habilidade de traçar está associada à habilidade de escrita?
- A habilidade de leitura (ou seja, "explicar em linguagem simples") está associada? Com habilidade de escrita?
- O desempenho do aluno neste exame é consistente com uma hierarquia de habilidades relacionadas à programação?

2. A AMOSTRA O exame foi

realizado por 78 alunos ao final do primeiro semestre de programação em Java. Trinta e oito desses alunos deram a aprovação institucional exigida para que seus trabalhos do exame fossem utilizados como dados de pesquisa. Esse subconjunto de alunos proporcionou uma distribuição razoável de notas, que refletia a distribuição da turma como um todo. A média das notas do subconjunto foi ligeiramente superior à da turma como um todo (62% contra 58%). Os alunos tiveram duas horas para concluir o exame de 28 páginas.

Os alunos frequentavam aulas teóricas comuns, mas eram divididos em turmas de laboratório de programação, ministradas por quatro tutores diferentes. A prova foi elaborada principalmente por dois desses tutores, com os demais fornecendo feedback e revisando o texto. A correção da prova foi feita em sistema de tiragem pelos quatro tutores — cada questão, em todas as provas, foi corrigida por um dos tutores e, posteriormente, moderada pelo restante da equipe docente.

3. O INSTRUMENTO O exame descrito aqui

é resultado de 3 anos de aprimoramento e desenvolvimento de avaliação baseado em pesquisa. Nesta seção do artigo, apresentamos as questões que compuseram o exame e também fornecemos estatísticas simples (por exemplo, média, quartis) que descrevem a distribuição de pontos para cada questão. A Tabela 1 apresenta o conjunto completo de estatísticas descritivas simples. O objetivo desta seção é familiarizar o leitor com as questões do exame antes da análise estatística mais sofisticada, incluindo a análise de caminhos, que será descrita posteriormente neste artigo.

Tabela 1: Estatísticas descritivas para cada questão da prova e a variável estatística à qual cada questão está atribuída.

Questão de exame nº.	Estatísticas Variável	Possível Marca	Média Marca (absoluto)	Média Marca (percentagem)	Terceiro Quartil	Mediana	Primeiro quartil
1.	Noções básicas	7	5.2	75%	6,8	6.0	4.0
2.	conceitos básicos	7	6.4	91%	7.0	7.0	6.3
3.	conceitos básicos	4	3.7	93%	4.0	4.0	3.6
4.	Princípios Básicos	8	5.3	66%	6,9	5,5	4.1
5.	Sequência	5	4.4	88%	5.0	4,5	4,5
6.	Sequência	6	4,9	83%	6.0	6.0	4.0
7.	Rastreamento A-Comer	15	10.1	67%	12,8	10.0	9.0
7.	Exceções Eii	2	0,9	47%	1.4	1.0	0,5
8.	Dados	10	7,8	78%	9.0	8.0	7.0
9.	Escrita	6	1.8	29%	3.6	0,8	0,0
10.	Explique	8	3.2	40%	4,8	3.0	2.0
11.	Escrita	6	2.7	45%	6.0	2.0	0,0
12.	exceções	6	1,5	25%	3.0	0,5	0,0
13.	Geral	10	4,7	47%	6.0	5.0	4.0
Total		100	62,7	63%	69,9	62,8	54,6

Tabela 2: Estatísticas descritivas para tipos de perguntas (ou seja, variáveis)

Média Marca Classificação	Tipo (ou variável)	Possível Marca	Média Marca (absoluto)	Média Marca (percentagem)	Terceiro Quartil	Mediana	Primeiro quartil
2	Noções básicas	26	20,6	79%	24,4	21.3	18,6
1 (mais fácil)	Sequência	11	9.3	85%	11.0	10,5	8,5
4	Rastreamento1 (não iterativo)	10	7,7	77%	10.0	8.0	6.0
5	Rastreamento 2 (iteração)	5	2.4	48%	3,8	2.0	1.0
9 (mais difícil)	Exceções	8	2.4	30%	4.0	1.8	0,6
3	Dados	10	7,8	78%	9.0	8.0	7.0
8	Escrita	12	4.4	37%	7.4	3,5	1.0
7	Explicar	8	3.2	40%	4,8	3.0	2.0
6	Em geral	10	4,7	47%	6.0	5.0	4.0

Antes de qualquer análise, os autores e colegas classificaram as 13 questões da prova em 8 categorias, com base em sua experiência docente e no trabalho anterior do projeto BRACElet. Essas 8 categorias são: *Conceitos Básicos*, *Sequência*, *Rastreamento*, *Exceções*, *Dados*, *Redação*, *Explicação* e *Geral*. Durante a análise, tornou-se evidente que a categoria *Rastreamento* deveria ser dividida em duas: *Rastreamento 1 (não iterativo)* e *Rastreamento 2 (iterativo)*.

3.1 Noções básicas (Questões 1-4)

Essas questões exigiam que os alunos identificassem construções da linguagem Java, reconhecessem a definição de termos comuns da linguagem e detectassem erros de sintaxe. Elas enfatizavam a memorização de conhecimentos prévios.

3.1.1 Questão 1: Associando Termos às Definições A primeira questão apresentou ao aluno 7 termos: *atribuição*, *compilador*, *construtor*, *depurador*, *método*, *sobrecarga* e *variável*. Também foram apresentadas 7 definições, sendo 2 delas “Traduz o código-fonte em código objeto” e “Código chamado quando um objeto é criado”. Os alunos deveriam associar os termos às definições.

Cada acerto valia 1 ponto. O desempenho geral dos alunos foi bom. A média foi de 5,2 em 7 (75%).

3.1.2 Questão 2: Correspondência de Termos com Código

Esta questão foi outro exercício de correspondência. Os alunos receberam uma página de código, uma definição completa de classe, com um único construtor, um único método de acesso, um único método modificador e um método "print" que usava System.out.println para exibir alguns membros de dados privados. Dez das linhas de código foram anotadas com um caractere alfabético, de "A" a "J". Os alunos também receberam 7 definições/descrições de código, incluindo "o nome de um método de acesso" e "a assinatura do construtor".

Os alunos tiveram que relacionar as definições/descrições às linhas de código anotadas de "A" a "J".

Cada acerto valia 1 ponto. No geral, os alunos se saíram bem nessa questão. A nota média foi de 6,1 em 7 (91%).

3.1.3 Questão 3: Cabeçalhos de Métodos

Esta questão apresentou aos alunos 4 cabeçalhos de métodos (ou seja, código Java). Os alunos deveriam fornecer o número de parâmetros e o tipo de retorno de cada cabeçalho. Cada cabeçalho valia 1 ponto. Os alunos tiveram um bom desempenho nesta tarefa, com uma nota média de 3,7 em 4 (93%).

3.1.4 Questão 4: Erros de Sintaxe

Nesta questão, os alunos deveriam encontrar 8 erros de sintaxe em uma página de código, que era uma definição completa de classe. Os alunos foram instruídos a encontrar 8 erros de sintaxe, e foram encontrados 11 erros no código. A classe continha 5 membros de dados privados, dos tipos String, double e ArrayList. Ela também continha um único construtor, 5 métodos de acesso/modificação simples e outro método que removía um elemento específico de um ArrayList.

Os alunos receberam 1 ponto para cada erro de sintaxe corretamente identificado. O desempenho dos alunos foi razoável nesta tarefa, especialmente considerando que havia 11 erros de sintaxe disponíveis, com uma nota média de 5,3 em 8 (66%).

3.1.5 Desempenho Geral nos Conceitos Básicos

No geral, os 38 alunos demonstraram domínio dessas tarefas "Básicas", com uma nota média de 21, em 26 (79%). Três quartos dos alunos obtiveram nota igual ou superior a 18 (69%). Em algumas outras questões apresentadas abaixo, o desempenho da turma como um todo é muito pior, e essas quatro primeiras questões sobre "Conceitos Básicos" comprovam que o baixo desempenho geral nas questões subsequentes não se deve a uma compreensão deficiente desses conceitos básicos.

3.2 Sequência (Questões 5-6)

3.2.1 Questão 5: Linhas Ausentes

Esta questão começa com um preâmbulo: "O código abaixo é de um dos projetos BlueJ que você usou neste semestre. Observe que ele usa um ArrayList. Parte do código foi removida." O código subsequente é uma definição de classe, ocupando uma página e meia, contendo dois membros de dados privados (um dos quais é um ArrayList), um único construtor e 3 métodos, que adicionam, removem e imprimem o conteúdo do ArrayList.

Abaixo do código, os alunos recebem a seguinte tarefa: "A tabela abaixo mostra as linhas de código que faltam, mas não necessariamente na ordem correta. Ela também contém uma linha de código extra que não é necessária." Identifique qual linha de código deve ir em qual lugar ...

Esta questão é semelhante às questões de código esqueleto do grupo de Leeds (Lister et al. 2004), mas no estudo de Leeds a tarefa era mais simples, pois geralmente havia uma única linha faltando no código e quatro opções alternativas eram fornecidas para essa linha faltante.

Por outro lado, como indicado no preâmbulo, os alunos já tinham visto esse código antes, o que facilitaria a tarefa.

Os alunos receberam 1 ponto por cada linha de código corretamente colocada nas 5 posições disponíveis. Eles se saíram muito bem nesta tarefa, com uma nota média de 4,4 em 5 (88%).

3.2.2 Questão 6: Quebra-cabeça de Parsons

Um Quebra-cabeça de Parsons (Parsons e Hayden, 2006) é o caso extremo do tipo de questão anterior. Ele exige que os alunos peguem um conjunto de linhas de código, apresentadas em ordem aleatória, e coloquem essas linhas na ordem correta para executar uma determinada função. O quebra-cabeça que usamos nesta prova é apresentado na Figura 1.

Esses quebra-cabeças exigem que os alunos apliquem seu conhecimento dos padrões comuns em algoritmos básicos, apliquem algumas heurísticas (por exemplo, inicializem uma variável antes de usá-la) e, talvez, também, em certa medida, demonstrem alguma habilidade de projeto. Parson e Hayden afirmaram que esses quebra-cabeças exigem habilidades entre leitura e escrita de código. Nenhum estudo empírico, até o momento, foi realizado para investigar essa afirmação.

Não havíamos utilizado o Quebra-Cabeça de Parsons em nenhuma prova anterior, nem havíamos mostrado aos alunos quaisquer exemplos desses quebra-cabeças antes da prova. O código utilizado nesta questão foi adaptado de uma questão de "explique em linguagem simples" de uma prova anterior, mas suspeitamos fortemente que poucos alunos tenham visto essa prova anterior antes de fazer esta.

Ficamos surpresos com o bom desempenho dos alunos nesta questão, com uma média de 4,9 em 6 (83%). Três quartos dos alunos obtiveram pelo menos 4 pontos, e uma nota de 4 implica que a resposta seria perfeita se duas linhas trocassem de lugar.

Em retrospectiva, dado o bom desempenho dos alunos nesta questão, eles podem ter conseguido posicionar corretamente muitas das linhas de código aplicando heurísticas superficiais. Por exemplo, a heurística "sempre coloque uma instrução 'return' no final" funcionaria neste caso. (No estudo do grupo de Leeds, os alunos tiveram dificuldade com uma questão em que não havia um "return" no final.) Além disso, ao fornecer a posição das chaves, podemos ter revelado informações demais, já que apenas as linhas A, B e E poderiam preceder as três chaves de abertura. Talvez a mesma questão tivesse sido mais difícil se não tivéssemos fornecido as chaves e, em vez disso, instruído os alunos a adicioná-las onde necessário.

3.3 Rastreamento1 – Não iterativo (Quest. 7, AC)

As partes A, B e C da questão 7 eram tarefas de traçado muito simples.

A Parte A (3 pontos) exigia que os alunos indicassem os valores de três variáveis após serem inicializadas e alteradas por três instruções de atribuição: a += 2; b -= 4; c = ba; A Parte B (4 pontos) exigia que os alunos calculassem o valor da expressão num1 % num2 para dois conjuntos diferentes de valores das duas variáveis. A Parte C (3 pontos) exigia que os alunos calculassem o valor da variável "bValid" para o seguinte código:

```
booleano bValid = falso;
```

```
se (iValue >= PRIMEIRO_VALOR && iValue < SEGUNDO_VALOR) {
```

```
    bValid = verdadeiro;
```

```
}
```

para três conjuntos diferentes de valores das variáveis, iValue, FIRST_VAL e SECOND_VAL.

Aqui estão alguns trechos de código que, quando usados na ordem correta, formariam um método para contar as ocorrências de uma letra em uma palavra (por exemplo, quantas vezes a letra 'm' aparece na palavra "Programming"?).

- A. se(sWord.charAt(i) == toCount)
- B. para(int i = 0; i < sWord.length(); i++)
- C. retornar contagem;
- D. int contagem = 0;
- E. public int contagemLetra(String sPalavra, char contagem)
- F. contagem++;

Cada caixa abaixo representa um espaço reservado para uma das linhas de código acima. Cada linha de código deve ser inserida em apenas uma das caixas. Indique qual linha de código deve ser inserida em qual caixa, escrevendo sua letra (de A a F) na caixa correspondente.

Figura 1: Questão 6, o Quebra-Cabeça de Parsons

A média geral das notas nessas três tarefas simples de traçado foi de 7,7 em 10 (77%), com três quartos dos alunos obtendo nota igual ou superior a 6. O desempenho geral foi mais baixo na parte B, talvez porque os alunos não conhecessem o operador "%". A média geral das notas nas partes A e C foi de 5,3 em 6 (88%), uma porcentagem média superior à de outros tipos de questões na Tabela 2.

3.4 Rastreamento2 – Iteração (Questões 7, DE(i))

3.4.1 Questão 7D: Rastreamento um laço while Os alunos receberam o código mostrado na Figura 2 e foram solicitados a fornecer o valor retornado pelo método para três valores diferentes do parâmetro "iLimit": -1, 3 e 0. O desempenho coletivo dos alunos foi mediano, com uma nota média de 1,8 em 3 (60%). Esse resultado está de acordo com as conclusões do Grupo de Leeds, que constatou que muitos alunos têm dificuldades com o rastreamento de laços.

3.4.2 Questão 7E(i): Laço for contendo um "if"

Os alunos receberam o código mostrado na Figura 3 e foram solicitados a fornecer o valor retornado pelo método, para dois casos.

diferentes valores do parâmetro "aNumbers", {1, 2, 3, 4, 5} e {20, -10, 6, -2, 0}. O desempenho coletivo dos alunos foi fraco, com uma nota média de 0,6 em 2 (32%).

Conforme demonstrado na Tabela 2, o desempenho dos alunos nessas duas tarefas de rastreamento de loop combinadas foi mediano, com uma nota média de 2,4 em 5 (48%).

```
público int q7D(int iLimit) {

    int Index = 0; int
    Result = 0;

    enquanto (iIndex <= iLimit) {

        Resultado += Índice;
        Índice++;

    } retornar iResultado;

}
```

Figura 2: Questão 7D, um loop while

```
public int q7E(int[] aNumbers) {

    int Resultado = 0;

    for(int iIndex=0; iIndex<aNumbers.length; iIndex++) {

        if(aNumbers[iIndex] > iResult) {

            Resultado = aNumbers[Índice];

        }

    } retornar iResultado;

}
```

Figura 3: Questão 7E(i), um laço for contendo um "if"

3.5 Exceções (Questão 7 E(ii) e 12)

Duas questões exigiam que os alunos demonstrassem alguma compreensão do conceito de exceção. A primeira delas, questão E(ii), utilizava o código fornecido na Figura 3 e perguntava aos alunos o que aconteceria se o método fosse chamado da seguinte forma: q7E(null); O desempenho coletivo dos alunos foi mediano, com uma nota média de 0,9 em 2 (47%).

A segunda questão sobre exceções, a questão 12, era composta por duas partes com o mesmo valor. A primeira fornecia aos alunos meia página de código, que manipulava uma estrutura de dados simples. Em seguida, os alunos eram informados: "O código contém um erro - ele compila e executa, mas às vezes para com uma exceção Index OutOfBoundsException.

Explique por que esse problema ocorre." Uma resposta adequada seria que as proteções "if... else if... else..." falharam quando a estrutura de dados estava cheia e houve uma tentativa de acessar o elemento mais alto da estrutura. A segunda parte da questão 12 era análoga à primeira, mas envolvia referências a objetos e uma NullPointerException. A maioria dos alunos deu respostas ruins em ambas as partes, com uma nota média de 1,5 de 6 (25%).

3.6 Dados (Questão 8)

Esta questão testou o conhecimento dos alunos sobre tipos de dados e consistia em quatro partes. Na primeira parte, os alunos foram apresentados a 5 tipos de dados: *ArrayList*, *Boolean*, *double*, *int* e

Os alunos também receberam 5 descrições de dados, incluindo “O nome de um aluno” e “Se uma pessoa é casada ou não”. Os alunos deveriam escolher o tipo de dado “mais apropriado” para cada descrição.

Na segunda parte da questão 8, os alunos foram solicitados a sublinhar o tipo de dados no código: `public Server host;`

Na terceira parte, os alunos foram solicitados a escrever “a declaração de uma variável que deve ser visível apenas dentro da turma atual, cujo nome seja representativo e que seja do tipo Aluno”.

Na quarta parte, os alunos foram testados quanto à sua compreensão de escopo. Foi fornecido a eles um trecho de código do livro didático, uma classe completa com cerca de meia página, contendo alguns membros de dados privados, um construtor e um acessador, chamada “showPrice”. Também foi informado que um pequeno erro havia sido introduzido no código e que “O código compila, mas não funciona como esperado. Qualquer valor passado para o construtor, quando showPrice é chamado, ele exibe: O preço de um ingresso é 0,0 centavos. Qual é o problema?” O erro era que, dentro do construtor, uma instrução de atribuição destinada a atualizar o membro de dados privado “price” estava incorretamente escrita como `double price = ticketCost;` portanto, a variável local “price” era atualizada, e não o membro de dados privado “price”.

Os 38 alunos demonstraram um sólido domínio da matéria, com uma média de 7,8 em 10 (78%). Três quartos dos alunos obtiveram nota igual ou superior a 7.

3.7 Redação (Questões 9 e 11)

3.7.1 *Questão 9: Escreva seu nome de Hip Hop.* Os alunos foram solicitados a fazer o seguinte: “Escreva um método em Java que gere seu nome de hip hop. Seu nome de hip hop começa com DJ.”

Em seguida, vem o nome do seu animal de estimação, seguido pelos 3 primeiros caracteres do seu primeiro nome e os 2 primeiros caracteres do nome de solteira da sua mãe.” Os alunos também receberam as seguintes informações sobre os métodos que poderiam usar para criar seu nome de Hip Hop: `char charAt(int index)` Retorna o valor `char` no

índice especificado.

- `int length()` Retorna o comprimento desta string.
- `String[] split(String regex)` Divide esta string em torno de correspondências da expressão regular fornecida.
- `String substring(int beginIndex)` Retorna uma nova string que é uma substring desta string.
- `String substring(int beginIndex, int endIndex)` Retorna uma nova string que é uma substring desta string.

Os alunos tiveram um desempenho ruim nesta tarefa, com uma nota média de 1,8 em 6 (29%).

3.7.2 *Questão 11: Driver de Teste da Força-Tarefa Zumbi* Os alunos receberam documentação, em formato JavaDoc, para os métodos que gerenciavam uma “Força-Tarefa” de instâncias da classe “Zumbi”, incluindo os seguintes métodos:

- `void addZombie(Zombie oNewMember)` Permite um Zumbi se juntará à força-tarefa.
- `int countZombies()` Conta o número de zumbis em a força-tarefa
- `void deleteZombie(Zombie oMember)` Permite um Zumbi será removido da força-tarefa.
- `booleano findZombie(java.lang.String sName)` Procure um zumbi pelo nome.

Em seguida, foi solicitado aos alunos que escrevessem uma única linha de código para executar cada uma das seguintes tarefas:

- Verifique se inicialmente não há zumbis no grupo.
- Adicione o zumbi chamado “Scrog”.
- Verifique se há apenas um zumbi no grupo.
- Verifique se “Scrog” pode ser encontrado no grupo.
- Verifique se “Blob” não pode ser encontrado no grupo.
- Remova “Scrog”

Verifique se agora não há zumbis no grupo.

O desempenho coletivo dos alunos foi medíocre, com uma nota média de 2,7 em 6 (45%).

3.7.3 *Correção e Exposição Prévia* Ao corrigirem essas duas questões de redação, os tutores foram instruídos a ignorar pequenos erros de sintaxe. A documentação do Java foi fornecida para quaisquer métodos da biblioteca Java que pudessem ser úteis aos alunos. A questão 9 era muito semelhante a um exercício dado aos alunos no mês anterior à prova. Testes unitários, como na questão 11, foram abordados ao longo do semestre. Apesar dessas circunstâncias favoráveis, a média geral dessas duas questões de redação foi de apenas 4,4 em 12 (37%). Um quarto dos alunos obteve nota igual ou inferior a 1.

3.8 Explique (Questão 10)

Esta questão era composta por 3 partes, valendo 2, 3 e 3 pontos, respectivamente. Em cada parte, os alunos receberam um código e foram instruídos a “explicar em linguagem simples o que ele faz”. O código das duas primeiras partes é mostrado nas Figuras 4 e 5, respectivamente. O código da terceira parte era mais extenso e implementava uma busca binária em um vetor de inteiros.

Diferentemente de estudos anteriores que utilizaram esse tipo de questão (Whalley et al., 2006; Lister et al., 2006), esses alunos já haviam visto esse tipo de questão antes da prova. Questões semelhantes foram apresentadas durante as aulas e em provas anteriores, com respostas-modelo. No entanto, os três trechos de código usados nesta prova não haviam sido apresentados aos alunos previamente, portanto, as respostas não poderiam ter sido memorizadas. Os alunos sabiam que uma descrição linha por linha do código (ou seja, uma resposta SOLO multiestrutural) resultaria em uma pontuação baixa, enquanto uma resposta com pontuação alta seria um resumo do cálculo geral realizado pelo código (ou seja, uma resposta SOLO relacional). O preâmbulo da questão 10 lembrava aos alunos: “Observe que mais pontos serão obtidos respondendo corretamente às questões anteriores”.

explicando a finalidade do código em vez de apenas descrever o que cada linha faz".

Apesar da experiência anterior, o desempenho dos alunos nessas questões foi medíocre, com a nota média agregada nas três partes sendo de apenas 3,2 em 8 (40%).

```
public double method10A(double[] aNumbers) {

    número duplo = 0;

    for(int iLoop = 0; iLoop < aNumbers.length; iLoop++) {

        num += aNumbers[iLoop];

    }

    retornar num;

}
```

Figura 4: o código para a Questão 10A, uma questão de leitura.

```
public void method10B(int iNum) {

    for(int iX = 0; iX < iNum; iX++) {

        for(int iY = 0; iY < iNum; iY++) {

            System.out.print("x");

        }

        System.out.println();

    }

}
```

Figura 5: o código para a Questão 10B, uma questão de leitura.

3.9 Geral (Questão 13)

A questão final do exame instruiu os alunos a "Selecionar UMA das seguintes opções e escrever uma resposta em inglês claro. Espera-se que a resposta tenha de 4 a 5 parágrafos". Os quatro tópicos que os alunos podiam escolher eram:

- "Meu código funciona, então deve ter sido bem escrito". Com referência aos conceitos de acoplamento e coesão, discuta como projetar programas que não apenas funcionem, mas também sejam fáceis de modificar.
- Arrays e ArrayLists são estruturas de dados que você estudou em Programação 1. Descreva as principais diferenças e semelhanças entre essas estruturas de dados. Serão considerados exemplos de onde você as utilizou em qualquer código que tenha escrito neste semestre.
- Por que precisamos testar o código?

Descreva como você testou o código que escreveu para sua tarefa neste semestre. Como os testes ajudaram você a escrever um código correto e funcional?

- Descreva os vários padrões e técnicas de formatação recomendados em Programação 1 e explique como cada um deles melhora a qualidade do código e torna o programador mais produtivo quando são utilizados.

Esta questão testa claramente conhecimentos e habilidades bastante diferentes das habilidades testadas em outras questões.

4. MÉTODO Na seção

anterior, as questões do exame/instrumento de pesquisa foram descritas em detalhes. A descrição incluiu estatísticas descritivas simples indicando a distribuição das notas dos alunos em cada questão e em cada tipo de questão. Nesta seção, apresentamos uma análise estatística mais sofisticada, que examina as relações entre as questões do exame e fornece evidências (a favor ou contra) uma hierarquia de habilidades relacionadas à programação.

4.1 Abordagem de Análise Quando

notas ou pontuações são atribuídas às questões, o avaliador geralmente confia nas propriedades ordinais das notas (ou seja, 4 é melhor que 3, 3 é melhor que 2, etc.). Há menos confiança nas propriedades intervalares das notas. Por exemplo, a diferença entre 3 e 4 representa a mesma diferença em conhecimento ou habilidades que a diferença entre 2 e 3? No entanto, a mensuração em nível intervalar é necessária para cálculos aritméticos válidos com variáveis.

Para abordar essa questão, utilizou-se um modelo Rasch politômico (Rasch, 1960; Andrich, 1978) para criar variáveis de nível intervalar a partir das notas; trata-se de um modelo estocástico que identifica as estimativas de máxima verossimilhança das localizações dos limiares de pessoa e item por meio da modelagem simultânea das estimativas de localização e da incerteza em sua localização. A Figura 6 mostra um mapa de densidade de probabilidade ilustrativo (para a questão 6) que apresenta a probabilidade de as notas 1, 2, 3, 4 e 6 serem atribuídas para qualquer habilidade dada, com a escala de habilidade padronizada para o intervalo de 0 a 10 e centrada em 5.

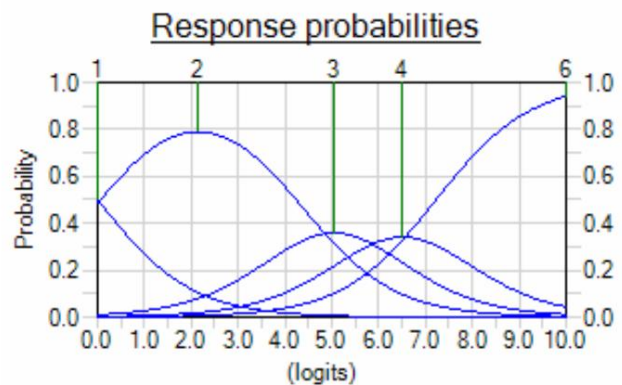


Figura 6: Probabilidades de resposta de amostra para a Questão 6.

Quebra-cabeça de Parsons

A partir deste diagrama, podemos ver que, para habilidades entre 0 e 4,7, a nota mais provável é 2; de 4,7 a 5,8, a nota mais provável é 3; de 5,8 a 6,6, é 4; e acima de 6,6, é 6. Este diagrama ilustra tanto os diferentes intervalos associados às notas quanto o padrão ordinal claro. Também podemos observar a incerteza nas notas (por exemplo, para uma habilidade imputada de 1, a nota mais provável é 2, mas há cerca de 30% de chance de ser atribuída a nota 1).

Uma premissa fundamental do modelo é que o construto que está sendo medido é unidimensional; isso foi verificado com uma análise convencional de componentes principais.

Existem muitas razões para a variabilidade nas notas dos exames; para examinar as potenciais relações estruturais entre os vários construtos, nosso primeiro passo foi tentar particionar a variabilidade no conjunto de dados entre aquela associada aos construtos de programação identificados e aquela associada a traços e atributos pessoais mais gerais. A variável geral (ou seja, a Questão 13) foi usada como uma estimativa desta última, estabelecendo um nível base com o qual outras estimativas poderiam ser comparadas. Escolhemos esta questão porque a capacidade de respondê-la não parece estar diretamente relacionada aos principais construtos examinados neste estudo. Deve-se notar, no entanto, que o uso de uma única questão, em vez de uma escala, não consegue capturar toda a gama de fatores pessoais.

Dividimos a variabilidade realizando um conjunto de regressões bivariadas entre a variável geral e todas as outras, e então usamos os coeficientes dessas regressões para subtrair o efeito geral de cada variável restante. Usar a variável geral como covariável dessa forma também melhora a generalidade, controlando o viés introduzido pela limitação da amostra àqueles que deram seu consentimento. Esse processo resultou em sete variáveis, cada uma representando a contribuição única feita por essa variável além do nível de base. As duas primeiras questões de pesquisa (ou seja, “A habilidade de traçar está associada à habilidade de escrita?” e “A habilidade de leitura (ou seja, explicar em linguagem simples) está associada à habilidade de escrita?”) foram então abordadas por uma hipótese de correlação positiva entre as variáveis apropriadas.

Para responder à terceira questão de pesquisa (ou seja, *podemos elaborar as relações entre esses e outros construtos?*), foi utilizada uma técnica de regressão múltipla stepwise. Nessa técnica, a escrita foi usada como variável critério e todas as outras variáveis foram inicialmente introduzidas no modelo como potenciais preditores. A cada passo, a variável com a menor contribuição única foi removida até atingirmos as três condições de parada: (1) R² ajustado máximo

, (2) uma regressão geral significativa (a 0,01) e (3) uma contribuição significativa (a 0,05) de cada variável.

Após identificar os preditores significativos da variância na escrita, o processo foi repetido, tomando cada um desses preditores como variável critério e buscando preditores significativos entre as variáveis restantes. Esse processo foi repetido para cada preditor identificado, a fim de construir um diagrama de caminhos com as possíveis cadeias de associação.

Os resíduos da regressão foram testados quanto à normalidade, homocedasticidade e independência serial com um teste de Jarque-Bera (Bera e Jarque, 1980).

5. RESULTADOS Nesta

seção, começamos apresentando os resultados do processo de triagem de dados e, em seguida, abordamos os resultados das questões de pesquisa. As alterações feitas nos procedimentos *a posteriori* são abordadas nas respectivas questões de pesquisa.

5.1 Análise dos dados Todos os

construtos foram unidimensionais, com exceção do traçado, que parece ter dois fatores subjacentes. A questão 7, partes A, B e C, apresentaram forte correlação com o primeiro fator, e as partes D e

E(i) no segundo fator. A principal diferença entre essas partes parece ser que as três primeiras partes envolveram o rastreamento de uma única passagem sequencial pelo código e as duas últimas envolveram uma estrutura de repetição. As cargas fatoriais, com a separação otimizada por uma rotação varimax, estão resumidas na Tabela 3.

Tabela 3: Rastreamento das cargas fatoriais

Papel	Característica	F1	F2
UM	sequência	0,5335	0,0579
B	sequência	0,6901	-0,0320
C	não iterativo	0,7617	0,2836
D	repetição (enquanto)	-0,0272	0,9030
E(i)	repetição (para)	0,1236	0,8691

Devido a esses dois fatores, foram criadas duas variáveis adicionais: Tracing1 (não iterativa) e Tracing2 (iteração), e essas duas foram utilizadas para a elaboração da terceira questão de pesquisa.

A variável “Geral” apresentou uma média de 48% e um desvio padrão de 29%. Apenas dois alunos receberam nota zero, o que sugere que, embora fosse a última questão da prova, a pressão do tempo não representou um grande obstáculo para os alunos que tentaram respondê-la.

Todas as variáveis derivadas apresentaram intercorrelações positivas, com exceção de “Dados” vs. “Explicação”, que apresentou uma correlação negativa pequena (mas não significativa). A intercorrelação média entre todas as variáveis foi de 0,3864.

5.2 Habilidade de traçado

Para abordar a proposição de que a habilidade de escrita de programas está associada à habilidade de traçado, testamos a hipótese de que a correlação entre traçado e escrita é positiva. Uma análise de correlação de Pearson produziu uma correlação de 0,5621 entre (Traçado) e (Escrita). Isso foi significativo no nível de 0,01 ($r(36)=0,5621$; $p=0,0001$) e positivo. A relação explica 32% da variabilidade; o R² ajustado foi de 30%. A equação de regressão é “**Escrita = 0,9728 • Traçado - 3,1803**”; o intervalo de confiança do coeficiente é: IC.99 = (0,3269 $\hat{y}$ Traçado $\hat{y}$ 1,6187). Um teste de normalidade de Jarque-Bera indica ($p=0,9794$) que a distribuição do resíduo da regressão está aceitavelmente próxima de uma distribuição normal, o que sugere que uma abordagem paramétrica é apropriada.

Como dois fatores subjacentes estavam associados à variável de traçado, testes separados foram realizados para cada um deles. Não foi encontrada correlação significativa ($r(36)=0,3028$; $p=0,0308$) entre Traçado1 (não iterativo) e Escrita. Houve uma correlação positiva significativa entre Traçado2 (iteração) e Escrita ($r(36)=0,6267$; $p<0,0001$). Essa relação explica 39% da variabilidade; o R² ajustado foi de 38%. Um teste de normalidade de Jarque-Bera indica ($p=0,3947$) que a distribuição dos resíduos da regressão está aceitavelmente próxima de uma distribuição normal, o que sugere que uma abordagem paramétrica é apropriada.

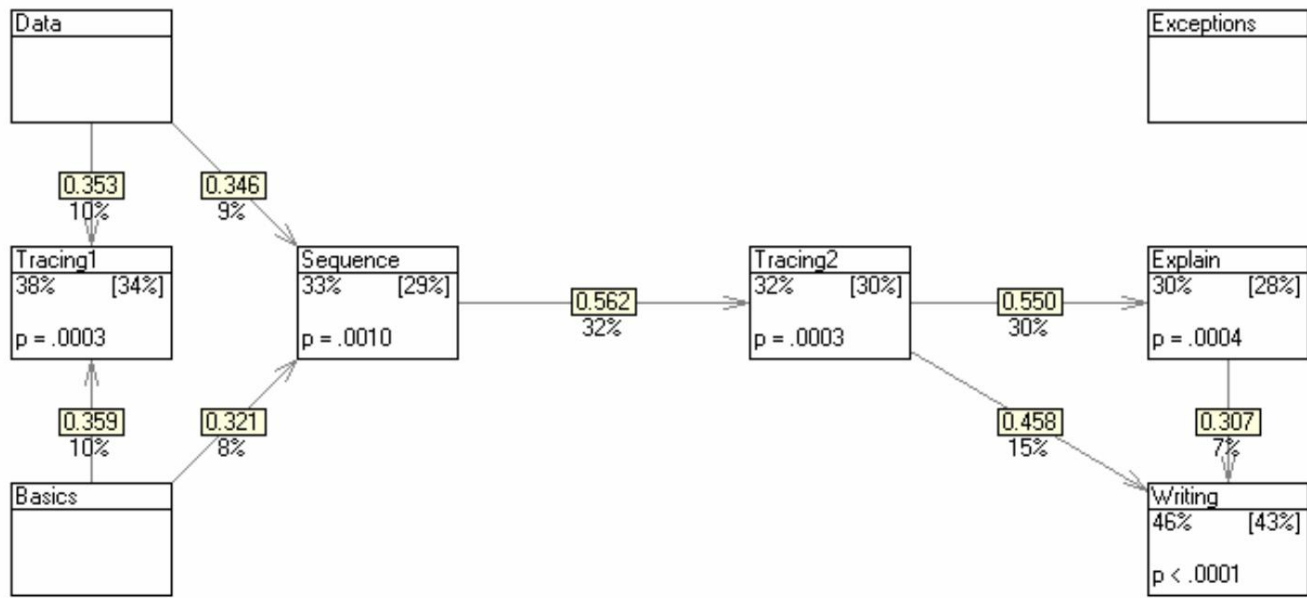


Figura 7: O Diagrama do Caminho

5.3 Habilidade de Leitura ("Explicação")

Para abordar a proposição de que a habilidade de escrever código está associada à habilidade de ler código, testamos a hipótese de que a correlação entre a capacidade de explicar código e a habilidade de escrever código era positiva. Uma análise de correlação de Pearson produziu uma correlação de 0,5586 entre (Explicação) e (Escrita). Essa correlação foi significativa no nível de 0,01 ($r(36)=0,5586$; $p=0,0002$) e positiva. A relação explica 31% da variabilidade; o R^2 ajustado foi de 29%. A equação de regressão é: "**Escrita = 0,6100 • Explicação + 1,1772**"; o intervalo de confiança do coeficiente é: IC.99 = (0,2013 y Explicação y 1,0187). Um teste de normalidade de Jarque-Bera indica ($p=0,3661$) que a distribuição do resíduo da regressão está aceitavelmente próxima de uma distribuição normal, o que sugere que uma abordagem paramétrica é apropriada.

5.4 Elaboração O procedimento

de regressão stepwise identificou as relações mostradas na Figura 7. Neste diagrama de caminhos, as variáveis são apresentadas em uma caixa com título contendo a variância explicada, o R^2 ajustado entre colchetes e a significância da regressão geral.

Os retângulos nos caminhos mostram os pesos beta dos caminhos, com o quadrado da correlação semiparcial apresentado abaixo. Este último representa a contribuição única para a explicação da variável critério, além daquela compartilhada com outras variáveis preditoras. A direção das setas não deve ser interpretada como evidência de causalidade (embora isso seja, obviamente, plausível).

É de se esperar um certo grau de multicolinearidade em qualquer instrumento de avaliação válido, mas isso tem como consequência o aumento do intervalo de confiança das estimativas da contribuição relativa de cada preditor para um critério, mesmo que a regressão geral permaneça estável. Neste conjunto de dados, o fator de inflação da variância atribuível à multicolinearidade na regressão variou de 1,30 a 1,43.

Embora seja um valor moderado, deve-se notar que, com esse tamanho de amostra, a contribuição relativa dos preditores está sujeita a considerável incerteza.

6. DISCUSSÃO Com relação às

duas primeiras questões de pesquisa abordadas neste estudo:

- Encontramos forte evidência de associação entre rastreamento de código e habilidades de escrita de código, particularmente quando o rastreamento envolvia loops (ou seja, Rastreamento2). A correlação entre Rastreamento2 e Escrita foi de 0,6267 ($p<0,01$), representando 39% da variabilidade, com um R^2 ajustado de 38%.
- Encontramos evidências que corroboram a associação entre a leitura de código (ou seja, "Explicar em linguagem simples") e as habilidades de escrita de código, com uma correlação de 0,5586 ($p<0,01$), representando 31% da variabilidade, e um R^2 ajustado de 29%.

6.1 Há evidências de uma hierarquia?

Esta subseção explora a terceira questão de pesquisa abordada neste estudo, analisando as evidências presentes neste exame que apontam para uma hierarquia de habilidades relacionadas à programação.

Como em qualquer análise estatística, as covariâncias identificadas neste estudo não constituem, por si só, evidência de causalidade. Contudo, se partirmos do pressuposto de um modelo causal, uma análise estatística subsequente poderá aumentar nossa confiança nesse modelo, manifestando covariâncias consistentes com ele. Antes de realizar a análise estatística (e, de fato, antes mesmo de elaborar a prova), tínhamos nossas próprias intuições sobre a estrutura da hierarquia de habilidades relacionadas à programação. Esta subseção explora se nossas intuições, e a literatura anterior, sobre tal hierarquia são consistentes com o diagrama de caminhos que emergiu da análise estatística.

6.1.1 Conceitos Básicos e Dados

Começamos na base da nossa hierarquia hipotética. Dada a influência da Taxonomia de Bloom revisada (Anderson et al., 2001), acreditamos que o conhecimento de construções básicas de programação constitui a base da hierarquia. Isso é consistente com a presença de “Conceitos Básicos” e “Dados” na parte inferior do diagrama de caminhos. Juntas, essas duas variáveis são fortes preditoras tanto das habilidades elementares de rastreamento (Rastreamento1) quanto das tarefas de Sequência.

6.1.2 Rastreamento 1 e Sequência

Ficamos surpresos ao constatar que não há uma relação estatisticamente significativa entre o Rastreamento 1 e o Rastreamento 2. Em retrospectiva, talvez as tarefas de Rastreamento 1 em sua prova fossem muito simples para demonstrar uma relação com o Rastreamento 2. Se for esse o caso, caracterizar a dificuldade de uma tarefa apenas por sua natureza pode ser insuficiente. A dificuldade de uma tarefa também pode ser função de seu tamanho e, talvez, também das construções de programação envolvidas.

Essa visão multidimensional da dificuldade pode ser consistente, em princípio, com a Taxonomia de Bloom revisada bidimensional (Anderson et al., 2001).

Antes da análise estatística, suspeitávamos que as tarefas de Sequência fossem de nível intermediário, e o diagrama de caminhos é consistente com essa suspeita. No entanto, ficamos surpresos ao ver que a variável Sequência apareceu em um nível inferior no diagrama de caminhos em comparação com a tarefa Tracing2, pois nossa intuição nos leva a crer que as tarefas de Sequência — tanto a conclusão do código-fonte quanto os Quebra-cabeças de Parsons — exigem um nível de habilidade superior ao do traçado. Contudo, como discutimos anteriormente, talvez o Quebra-cabeça de Parsons específico que utilizamos fosse muito fácil. Conforme mencionado no parágrafo anterior, em relação às tarefas Tracing1 e Tracing2, talvez existam outras características da tarefa, além de sua natureza, que determinem seu nível de dificuldade.

6.1.3 Rastreamento2 e Explicação

A relação entre Rastreamento2 e Explicação no diagrama de caminho é consistente com a conjectura do “semáforo” de Philpott, Robbins e Whalley (2007) y a luz verde para o pensamento relacional é o domínio do rastreamento de código.

Em conjunto, as habilidades Tracing2 e Explain explicam 46% da variância na escrita (uma porcentagem considerada boa por muitos nas ciências sociais, dada a preponderância de outros fatores nas vidas complexas das pessoas). Isso está de acordo com nossa intuição anterior e também com a literatura prévia, de que Tracing e Explain são habilidades intermediárias. No entanto, ficamos surpresos que Explain, sozinha, explique uma porcentagem menor da variância na escrita do que Tracing2 sozinha. Novamente, talvez existam outras características de uma tarefa, além de sua natureza, que determinem seu nível de dificuldade. Além disso, talvez as habilidades relacionadas à programação não formem uma hierarquia rígida, mas sim que habilidades de mais de um nível inferior possam influenciar o desempenho de uma habilidade em algum nível superior da hierarquia. E talvez, de forma mais prosaica, as tarefas de leitura em nosso exame não tenham sido tarefas de avaliação muito eficazes.

6.1.4 Exceções e outras questões

Em retrospectiva, não nos surpreende que Exceções esteja desconectada no diagrama de caminhos. As tarefas de escrita específicas desta prova não exigiam uma compreensão de exceções.

Este exame é um reflexo dos nossos tempos híbridos, com sua mistura de conceitos orientados a objetos e estruturas de controle clássicas de linguagens de terceira geração.

As tarefas de escrita enfatizaram mensagens para objetos (ou chamadas de procedimento, em termos de 3GL), mas tanto as tarefas de Rastreamento 2 quanto as de Explicação enfatizaram a compreensão do código que ocorre dentro dos métodos, particularmente as estruturas de controle. Pode ser surpreendente, portanto, que o Rastreamento 2 e a Explicação expliquem 46% da variância na escrita. Talvez a forte conexão combinada das tarefas de Rastreamento 2 e Explicação com as tarefas de escrita seja menos um reflexo de conhecimento específico de programação e mais um reflexo das habilidades genéricas de raciocínio exigidas nessas tarefas.

7. CONCLUSÃO

Para a maioria

dos acadêmicos, uma prova é um instrumento para atribuir notas aos alunos. Da perspectiva da pesquisa em educação computacional, uma prova também é um instrumento de pesquisa, uma oportunidade para estudar o conhecimento, as habilidades e a aprendizagem de iniciantes. Neste artigo, utilizamos uma prova como instrumento de pesquisa para explorar o conceito de uma hierarquia de habilidades relacionadas à programação. Encontramos evidências estatísticas que corroboram nossa intuição prévia e a literatura existente sobre a estrutura dessa hierarquia.

Nossa análise também apresentou resultados inesperados, como indícios de que resolver os Quebra-Cabeças de Parsons pode exigir uma habilidade menor do que rastrear código iterativo. No entanto, outras características de uma tarefa, além de sua natureza, podem determinar seu nível de dificuldade. À medida que nós (e esperamos que outros) elaborarmos e analisarmos exames futuros, essas questões e anomalias serão resolvidas.

Existem limitações no uso de exames como instrumento de pesquisa?

Por exemplo, para melhor compreender a relação entre tarefas de “explicação em linguagem simples” e escrita de código, talvez seja necessário que cada aluno complete 20 tarefas de explicação e escreva 20 pequenos trechos de código. Isso não é possível em uma prova, dada a sua curta duração e a necessidade de avaliar os alunos em um amplo espectro de conhecimentos e habilidades. Mas isso levanta um dilema interessante na interseção entre ensino e pesquisa: se uma prova é curta demais para ser um instrumento válido para pesquisar os alunos, ela ainda pode ser um instrumento válido para avaliá-los?

8. AGRADECIMENTOS

Os autores agradecem a

Tony Clear e aos demais colegas do projeto BRACElet. Raymond Lister é membro associado do Conselho Australiano de Ensino e Aprendizagem (ALTC).

No entanto, as opiniões expressas neste artigo não representam necessariamente as opiniões da ALTC.

9. REFERÊNCIAS

- Adelson, B. Quando os novatos superam os especialistas: A dificuldade de uma tarefa pode aumentar com a experiência. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 10, 3 (1984), 483-495.
- Anderson, LW, Krathwohl, DR, Airasian, PW, Cruikshank, KA, Mayer, RE, Pintrich, PR, Rath, J., & Wittrock, MC (eds). (2001). *Uma taxonomia para aprendizagem, ensino e avaliação: Uma revisão da taxonomia de objetivos educacionais de Bloom*. Nova York: Addison Wesley Longman Inc.
- Andrich, D. (1978). Uma formulação de classificação para categorias de resposta ordenadas. *Psychometrika*, 43, 561-73.
- Bera, A., Jarque, C. (1980), “Testes eficientes para normalidade,

- homocedasticidade e independência serial dos resíduos de regressão". *Economics Letters* 6 (3): 255–259.
doi:10.1016/0165-1765(80)90024-5.
5. Biggs, JB, & Collis, KF (1982). *Avaliando a qualidade da aprendizagem: A taxonomia SOLO (Estrutura do Resultado de Aprendizagem Observado)*. Nova York: Academic Press.
6. Corritore, C. e Wiedenbeck, S. O que os novatos aprendem durante a compreensão do programa? *Int. J. of Human-Computer Interaction*, 3, 2 (1991), 199-222.
7. Biggs, JB & Collis, KF Avaliando a qualidade de Aprendizagem: A taxonomia SOLO (Estrutura do Resultado de Aprendizagem Observado). Nova Iorque, Academic Press, 1982.
8. Lister, R. Sobre o florescimento da programação no primeiro ano e sua avaliação. *Anais da Conferência Australiana de Educação em Ciência da Computação*. ACM Press, 2000, 158-162.
9. Lister, R., Simon, B., Thompson, E., Whalley, J. e Prasad C. Não enxergando a floresta por causa das árvores: programadores novatos e a taxonomia SOLO. *ACM SIGCSE Bulletin*, 38(3), 2006, 118-122
10. Parsons, D. e Haden, P. (2006). *Parson's Programming Puzzles: A Fun and Effective Learning Tool for First Programming Courses*. In Proc. Eighth Australasian Computing Education Conference (ACE2006), Hobart, Australia. CRPIT, **52**. Tolhurst, D. e Mann, S., Eds., ACS. 157-163.
11. Philpott, A., Robbins, P. e Whalley, J. Acessando o Passos no Caminho para o Pensamento Relacional. *Anais da 20ª Conferência Anual da NACCQ*. Mann, S e Bridgeman, N. (orgs.), Nelson, Nova Zelândia, 8 a 11 de julho de 2007, p. 286.
12. Rasch, G. (1960/1980). *Modelos probabilísticos para alguns testes de inteligência e desempenho*. (Copenhague, Instituto Dinamarquês de Pesquisa Educacional), edição ampliada (1980) com prefácio e posfácio de BD Wright.
Chicago: Imprensa da Universidade de Chicago.
13. Soloway, E. e Ehrlich, K, Bonar, J., e Greenspan, J. (1983) *O que os novatos sabem sobre programação?* Em B. Shneiderman e A. Badre, editores, *Directions in Human-Computer Interactions*, 27–53. Ablex, Inc., Norwood, NJ.
14. Spinellis, D. Leitura, escrita e código. *ACM Queue*, 1(7), 2003, 84 - 89.
15. Thompson, E., Whalley, J., Lister, R., Simon, B. (2006) *Classificação de código como exercício de aprendizagem e avaliação para programadores iniciantes*. Anais da 19ª Conferência Anual do Comitê Consultivo Nacional de Qualificações em Computação (NACCQ), Wellington, Nova Zelândia, 7 a 10 de julho. pp. 291-298.
<http://bitweb.tekotago.ac.nz/staticdata/allpapers/2006/paperp/291.pdf>
16. Thompson, E., Luxton-Reilly, A., Whalley, J., Hu, M., & Robbins, P. (2008). *Taxonomia de Bloom para avaliação de CS*. Anais da Décima Conferência Australiana de Educação em Computação (ACE2008), Wollongong, Austrália, janeiro de 2008.
17. Whalley, JL, Lister, R., Thompson, E., Clear, T., Robbins, P., Kumar, PKA, & Prasad, C. (2006). Um estudo australiano sobre habilidades de leitura e compreensão em programadores iniciantes, utilizando as taxonomias de Bloom e SOLO. *Anais da Oitava Conferência Australiana de Educação em Computação (ACE2006)* (pp. 243-252). Hobart, Austrália: CRIPT.
18. Whalley, J, Clear, T e Lister, R. (2007), As muitas maneiras do projeto BRACElet. *Boletim de Aplicações . Computação e Tecnologia da Informação (BACIT)* Vol. 5, Edição 1. ISSN 1176-4120. http://www.naccq.co.nz/bacit/0501/2007Whalley_BRACELET_Ways.htm
19. Whalley, J e Robbins, P (2007) Relatório sobre o Quarto Workshop BRACElet. *Boletim de Computação Aplicada e Tecnologia da Informação (BACIT)* Vol. 5, Edição 1. ISSN 1176-4120. http://www.naccq.ac.nz/bacit/0501/2007Whalley_BRACELET_Workshop.htm
20. Wiedenbeck, S., Fix, V. & Scholtz, J. Características das representações mentais de programadores novatos e especialistas: um estudo empírico. *International Journal of Man-Machine Studies*, 39 (1993) 793-812.